

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Nguyễn Toàn Thắng - 25521679

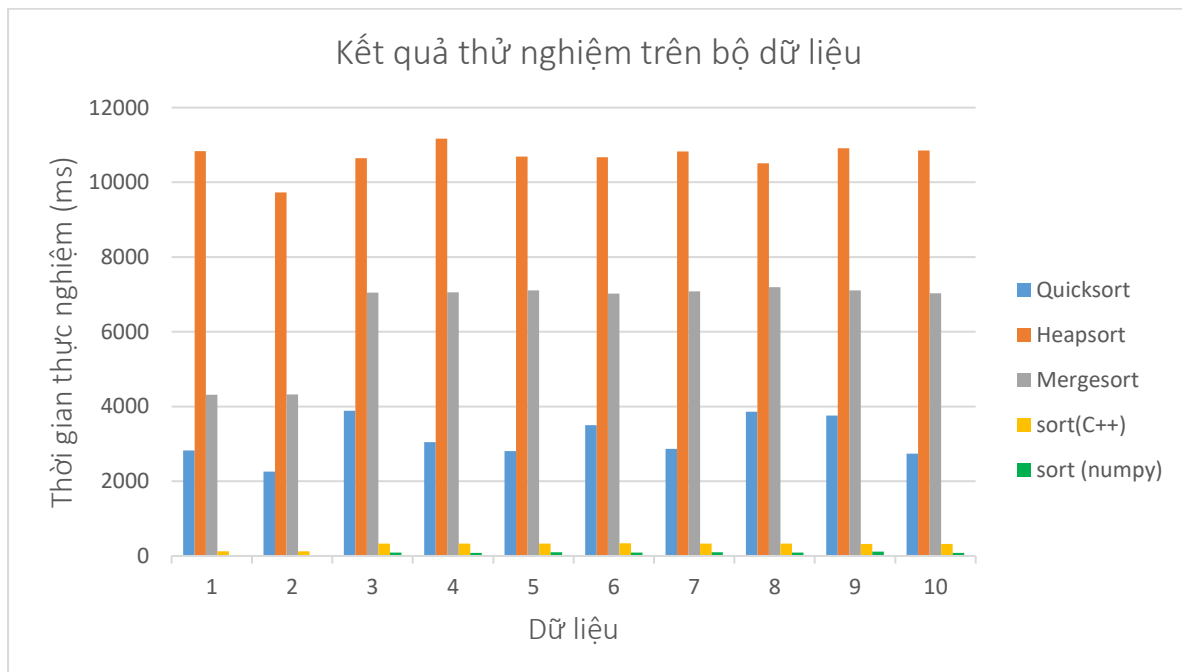
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

Dữ liệu	Thời gian thực hiện (ms)				
	Quicksort	Heapsort	Mergesort	sort (C++)	sort (numpy)
1	2819.96	10834.42	4310.44	123.83	13.91
2	2258.74	9729.42	4324.47	127.61	18.35
3	3888.03	10650.85	7045.72	327.90	88.87
4	3042.57	11168.14	7054.72	334.57	85.84
5	2807.62	10693.40	7106.75	328.35	94.84
6	3499.45	10672.21	7018.63	336.50	90.07
7	2867.54	10826.21	7078.48	328.96	95.77
8	3861.33	10507.32	7194.83	327.29	87.53
9	3759.70	10911.46	7104.18	324.51	118.09
10	2737.61	10855.01	7032.98	324.45	86.17
Trung bình	3154.26	10684.84	6527.12	288.40	77.94

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

¹ Số liệu chỉ mang tính minh họa

Từ kết quả thực nghiệm, ta rút ra được những nhận xét và kết luận như sau:

- **Sự chênh lệch về tốc độ (thời gian thực hiện):**

- **Nhóm dẫn đầu (Hiệu suất cao):** **sort (numpy)** đứng đầu tuyệt đối về tốc độ với thời gian trung bình chỉ 77.94 ms, theo sát là **sort (C++)** với 288.40 ms.
 - ⇒ Numpy đạt tốc độ nhanh nhất vì thực chất nó gọi các hàm sắp xếp đã được tối ưu hóa cực hạn bằng ngôn ngữ C/Fortran bên dưới. C++ nhanh vì được biên dịch trực tiếp ra mã máy, loại bỏ bước trung gian và tối ưu hóa việc quản lý bộ nhớ thủ công.
- **Nhóm trung bình (Tự cài đặt):** Vị trí thứ 3 và thứ 4 lần lượt thuộc về **Quicksort** (3154.26 ms) và **Mergesort** (6527.12 ms). Qua đó, ta cũng thấy được là **Quicksort** cho thấy ưu thế vượt trội khi nhanh gấp đôi **Mergesort**.
 - ⇒ Trong khi **Quicksort** hiệu quả nhờ cơ chế phân vùng tốt, đặc biệt nhanh khi dữ liệu đã có sẵn thứ tự, giúp giảm thiểu số phép hoán đổi cần thiết, thì **Mergesort** ổn định nhưng tốn thời gian cho việc cấp phát bộ nhớ để tạo các mảng phụ và thao tác trộn (merge) liên tục.
- **Nhóm thấp nhất: Heapsort** có thời gian thực hiện cao nhất trong tất cả các thử nghiệm với trung bình 10684.84 ms (xấp xỉ 10.7 giây).
 - ⇒ **Heapsort** chậm nhất do cách truy cập dữ liệu không tuần tự trên cấu trúc cây (Heap), khiến CPU thường xuyên gặp lỗi vì không tìm thấy dữ liệu sẵn trong bộ nhớ đệm, làm tăng thời gian đọc RAM.

- **Biến động theo tính chất dữ liệu:**

- Mặt khác, ta thấy tất cả các thuật toán đều đạt tốc độ tốt nhất trên dãy đã tăng hoặc giảm dần. Điển hình là **Quicksort** giảm xuống mức ~2200-2800 ms. Điều này minh chứng việc chọn *pivot* ở giữa mảng đã giúp thuật toán tránh được trường hợp xấu nhất ($O(n^2)$).
- Ngược lại, các bộ dữ liệu từ số 3 đến số 10 (ngẫu nhiên) chứng kiến thời gian thực hiện tăng vọt và duy trì ở mức cao ổn định, phản ánh đúng độ phức tạp thực tế khi xử lý các dãy số lộn xộn. Đặc biệt, **Heapsort** luôn ở mức trên 10.000 ms, thể hiện sự kém thích nghi của cấu trúc cây nhị phân khi phải xử lý dữ liệu không có thứ tự trong môi trường thông dịch.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

https://github.com/Meow-advocate/25521679_DSA_1.git